HIGH-LEVEL DESIGN (HLD)

RAG-Based Customer Support Assistant using LangGraph & HITL

1. System Overview

Problem Definition

Traditional customer support systems:

- Provide generic responses
- Lack contextual understanding
- Cannot handle document-based queries effectively

There is a need for a system that:

- Understands user queries
- Retrieves relevant information from documents
- Generates accurate and contextual responses
- Handles uncertain queries intelligently

Scope of the System

This system is designed to:

- Process a PDF knowledge base (Sales & Distribution Management)
- Retrieve relevant information using embeddings
- Generate contextual answers
- Implement workflow-based decision logic
- Support Human-in-the-Loop (HITL) escalation

Component Description

1. Document Loader

- Loads PDF using PyPDFLoader
- Extracts raw textual data

2. Chunking Strategy

- Splits document into smaller chunks

- Uses:
    - Chunk size: 500
    - Overlap: 100

3. Embedding Model

- Converts text into vector format
- Uses:
    - HuggingFace (all-MiniLM-L6-v2)

4. Vector Store

- Uses **ChromaDB**
- Stores embeddings for efficient similarity search

5. Retriever

- Performs similarity search
- Returns top-k relevant chunks

6. LLM (Language Model)

- Uses HuggingFace model (Flan-T5)
- Generates answers based on retrieved context

7. Graph Workflow Engine (LangGraph)

- Controls system flow
- Implements:
    - Input → Processing → Output
    - Conditional routing

8. Routing Layer

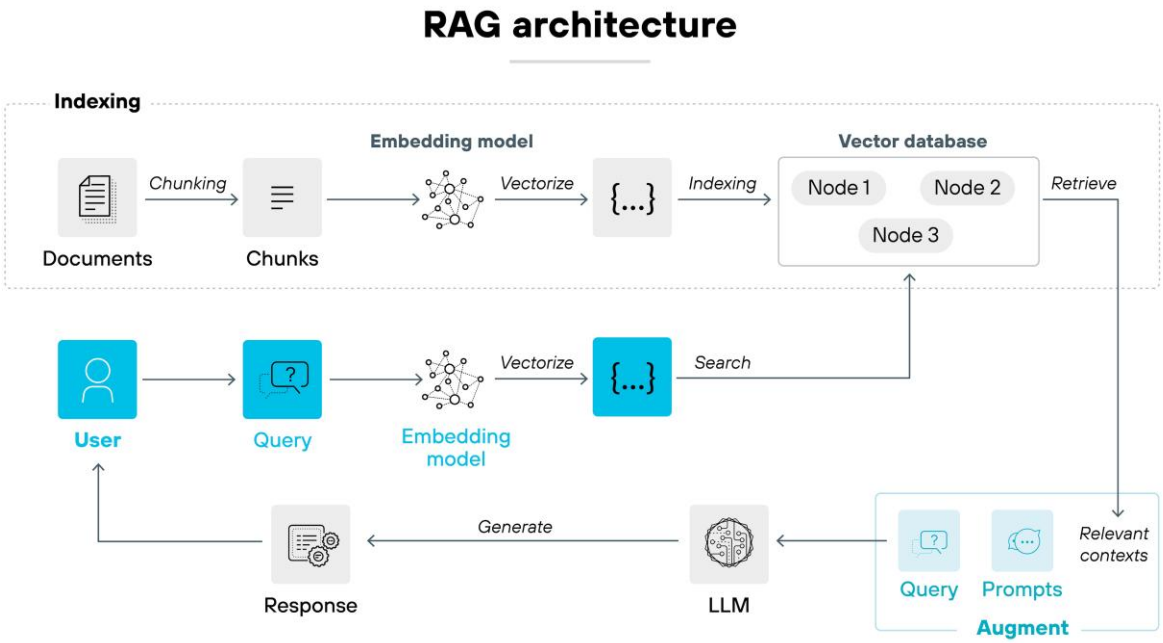
- Decides:
    - Generate answer OR
    - Escalate to human

9. HITL Module

- Handles human intervention

- Activated when:
    - Low confidence
    - No relevant data

Architecture Diagram



4. Data Flow

PDF how Answer Flow

1. PDF is loaded
2. Text is chunked
3. Chunks converted to embeddings
4. Stored in ChromaDB
5. User query received
6. Query converted to embedding
7. Relevant chunks retrieved
8. Passed to LLM
9. Answer generated

10. Decision layer checks confidence
11. If low confidence → HITL triggered

Query Lifecycle

User Query → Embedding → Retrieval → Context → LLM → Response → Decision → Output

5. Technology Choices

ChromaDB

- Lightweight vector database
- Easy integration
- Fast similarity search

LangGraph

- Enables workflow-based architecture
- Supports conditional routing
- Better control over execution

HuggingFace Models

- Free and open-source
- No API cost
- Good performance for small-scale systems

Python

- Rich ecosystem for AI/ML
- Easy integration with libraries

6. Scalability Considerations

Handling Large Documents

- Efficient chunking strategy
- Use of vector database for indexing

Increasing Query Load

- Can scale using:
    - Caching

    - Parallel processing
    - Cloud deployment

Latency Optimization

- Use smaller embedding models
- Optimize chunk size
- Limit top-k retrieval

Conclusion

The proposed system provides a scalable and efficient architecture for building a RAG-based customer support assistant. By integrating retrieval, generation, and workflow-based decision-making, the system ensures accurate, reliable, and context-aware responses while supporting human intervention when necessary.